

NumPy・Pandasと機械学習の基礎

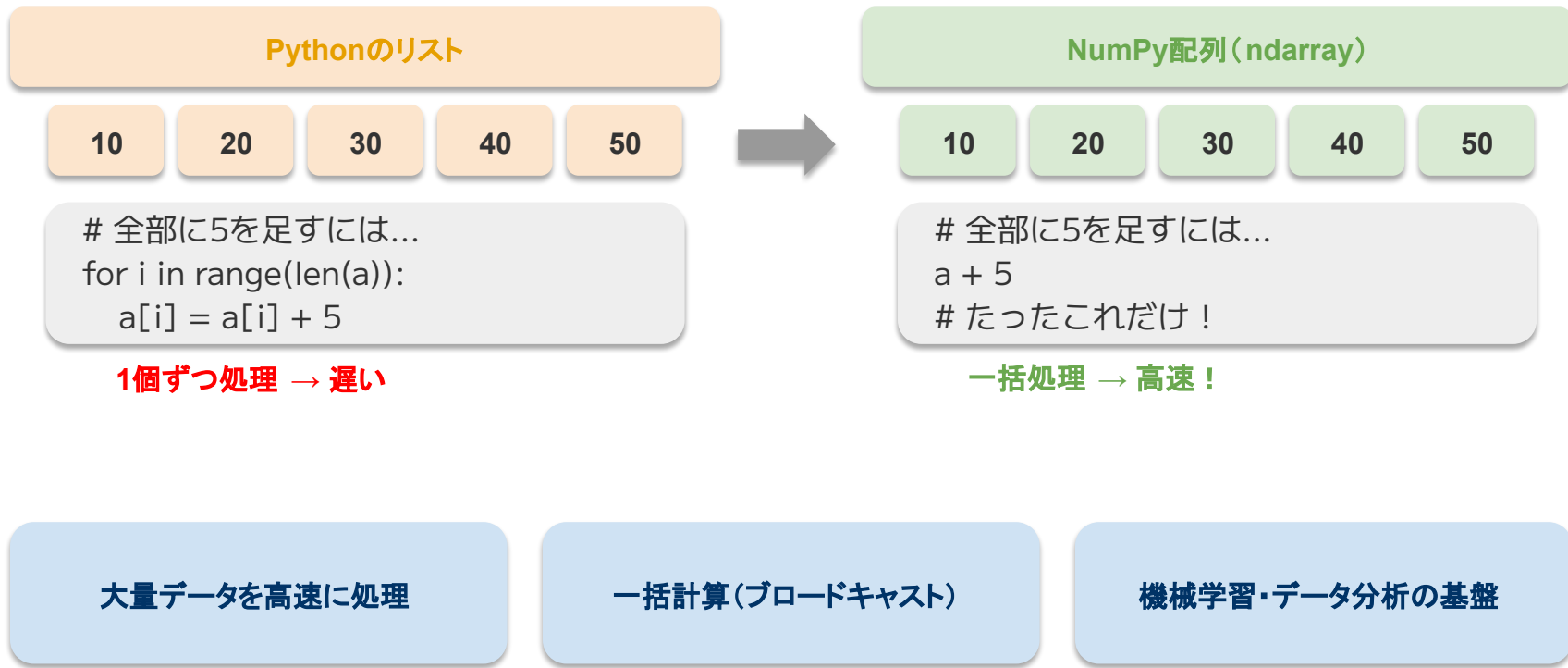
超サポ
愉快カンパニー

アシスト

NumPyとは

NumPyの概要

- NumPy(ナムパイ)は、Pythonで数値計算を効率的に行うためのライブラリです



NumPyのインポート

- 以下の灰色の枠内のコマンドを同じように書いてみましょう

```
import numpy as np
```

```
# 「numpy」を「np」という短い名前で使えるようにする
```

```
# これはNumPyを使うときのお約束です
```

配列 (array) の作成

- NumPyの基本は「配列」です。リストに似ていますが、計算に強い構造です

```
import numpy as np

# リストから配列を作る
a = np.array([1, 2, 3, 4, 5])
print(a)      # [1 2 3 4 5]
print(type(a)) # <class 'numpy.ndarray'>
```

np.array() とは

Pythonのリストを
NumPy配列に変換します

※ NumPy配列は ndarray (N-dimensional array) と呼ばれます

便利な配列の作り方

- よく使う配列の生成方法を覚えましょう

```
import numpy as np

zeros = np.zeros(5)      # [0. 0. 0. 0. 0.]
ones  = np.ones(3)       # [1. 1. 1.]
rng   = np.arange(0, 10, 2) # [0 2 4 6 8]
lin   = np.linspace(0, 1, 5) # [0. 0.25 0.5 0.75 1. ]
```

np.zeros / np.ones

0 や 1 で埋めた配列を作る

np.arange / np.linspace

等間隔の数列を生成する

配列の演算(要素ごとの計算)

- NumPy配列では、すべての要素に一括で計算ができます

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])

print(a + 5)  # [15 25 35 45 55]
print(a * 2)  # [ 20 40 60 80 100]
print(a / 10) # [1. 2. 3. 4. 5.]
```

ポイント

リストでは for 文が必要な
処理を一行で書ける
これを「ブロードキャスト」
と呼びます

演習: NumPyを使った計算

- 以下の問題をやってみましょう

```
# テストの点数データ
```

```
scores = np.array([65, 80, 55, 90, 72])
```

```
# 問題1: 全員に5点加算した結果を表示しなさい
```

```
# 問題2: 平均点を求めなさい (ヒント: np.mean())
```

```
# 問題3: 最高点と最低点を求めなさい (ヒント: np.max(), np.min())
```


演習: 回答

- 正解は以下のとおりです

```
import numpy as np
scores = np.array([65, 80, 55, 90, 72])

# 問題1: 全員に5点加算
print(scores + 5)    # [70 85 60 95 77]

# 問題2: 平均点
print(np.mean(scores)) # 72.4

# 問題3: 最高点・最低点
print(np.max(scores))  # 90
print(np.min(scores))  # 55
```

Pandasとは

Pandasの概要

- Pandas(パンドス)は、表形式のデータを扱うためのライブラリです

Excelの表(イメージ)

名前	数学	英語
田中	80	70
佐藤	65	85
鈴木	90	75



DataFrame(Pandasの表)

名前	数学	英語
田中	80	70
佐藤	65	85
鈴木	90	75

S

Series (1列のデータ)

1次元のデータ。Excelの「**1列**」に相当。
例: [80, 65, 90], [70, 85, 75]

DF

DataFrame (表形式データ)

行と列がある2次元の表データ。
Excelのシート1枚分のイメージ。

CSV読み込みが簡単

集計・フィルタリングが得意

Excelライクな操作感

Series (1列のデータ)

- 以下の灰色の枠内のコマンドを同じように書いてみましょう

```
import pandas as pd
```

```
# Seriesを作る
```

```
s = pd.Series([170, 165, 180, 155])
```

```
print(s)
```

```
# インデックス (ラベル) を付ける
```

```
s = pd.Series([170, 165, 180, 155],  
              index=["田中", "佐藤", "鈴木", "高橋"]) #indexが行名  
print(s)
```

pd.Series() とは

名前付きの1列データ

Excel の 1列分に相当

DataFrame(表形式のデータ)

- DataFrameは行と列を持つ表データです。辞書から作れます

```
import pandas as pd

data = {
    "名前": ["田中", "佐藤", "鈴木"],
    "数学": [80, 65, 90],
    "英語": [70, 85, 75]
}
df = pd.DataFrame(data)
print(df)
```

ポイント

辞書のキー → 列名

辞書の値 (リスト) → 各列のデータ

※ DataFrameは最もよく使うPandasのデータ構造です

データの確認メソッド

- DataFrameの中身を確認するための便利なメソッドを覚えましょう

先頭5行を表示

```
df.head()
```

データの基本情報（列名、型、欠損値の数）

```
df.info()
```

統計量（平均、標準偏差、最小最大など）

```
df.describe()
```

行数と列数

```
print(df.shape) # (3, 3)
```

データの選択とフィルタリング

- 特定の列や条件に合うデータだけを取り出すことができます

```
# 列を選択
```

```
print(df["名前"])
```

```
# 条件でフィルタリング
```

```
print(df[df["数学"] >= 80])
```

```
# 複数条件 (&: かつ、 |: または)
```

```
print(df[(df["数学"] >= 80) & (df["英語"] >= 70)])
```

ポイント

Day2で学んだif文の条件と同じ比較演算子を使います
& は and、| は or に相当

演習: DataFrameの操作

- 以下の問題をやってみましょう

```
import pandas as pd
data = {
    "名前": ["田中", "佐藤", "鈴木", "高橋", "伊藤"],
    "年齢": [25, 30, 22, 35, 28],
    "部署": ["営業", "開発", "営業", "開発", "人事"]
}
df = pd.DataFrame(data)

# 問題1: 年齢が25以上の人を抽出しなさい
# 問題2: 部署が"開発"の人を抽出しなさい
# 問題3: 年齢の平均を求めなさい (ヒント: df["年齢"].mean())
```


演習: 回答

- 正解は以下のとおりです

```
# 問題1: 年齢が25以上  
print(df[df["年齢"] >= 25])
```

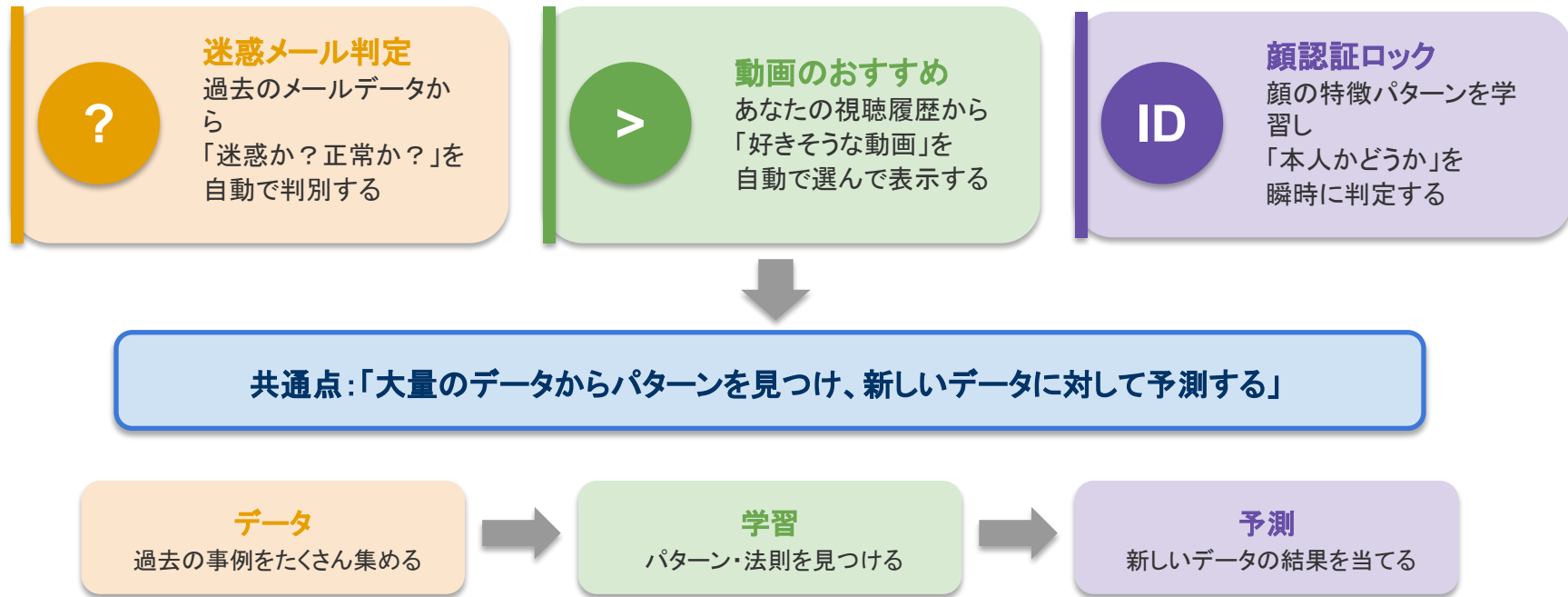
```
# 問題2: 部署が"開発"  
print(df[df["部署"] == "開発"])
```

```
# 問題3: 年齢の平均  
print(df["年齢"].mean()) # 28.0
```

機械学習とは

日常生活における機械学習

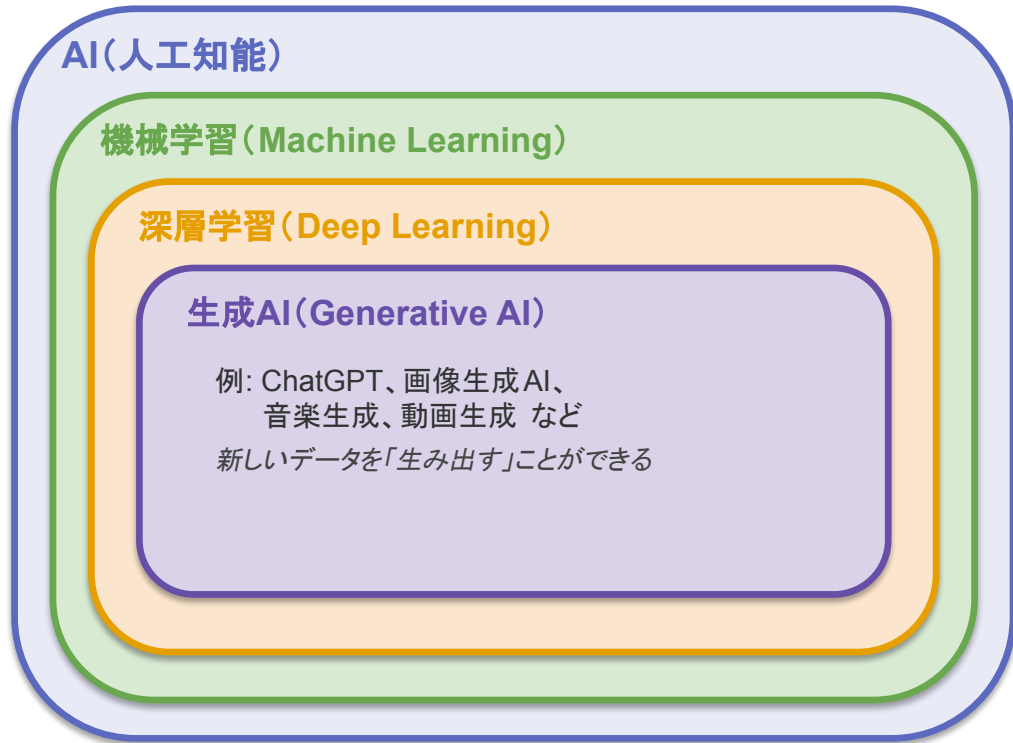
- 機械学習は、私たちの身の回りでたくさん活用されています



※ これが「機械学習」の基本的な考え方です

AI・機械学習・深層学習・生成 AIの関係

- これらの用語は入れ子の関係になっています。外側ほど広い概念です



※ 今日の授業では「機械学習」の部分学びます

AI

AI(人工知能)

人間の知能を模倣する技術全般。
ルールベース・検索・推論なども含む

ML

機械学習

データからパターンを自動で学習する。
今日学ぶ scikit-learn はここ！

DL

深層学習

ニューラルネットワークで複雑な
パターンを学習。画像・音声認識に強い

G

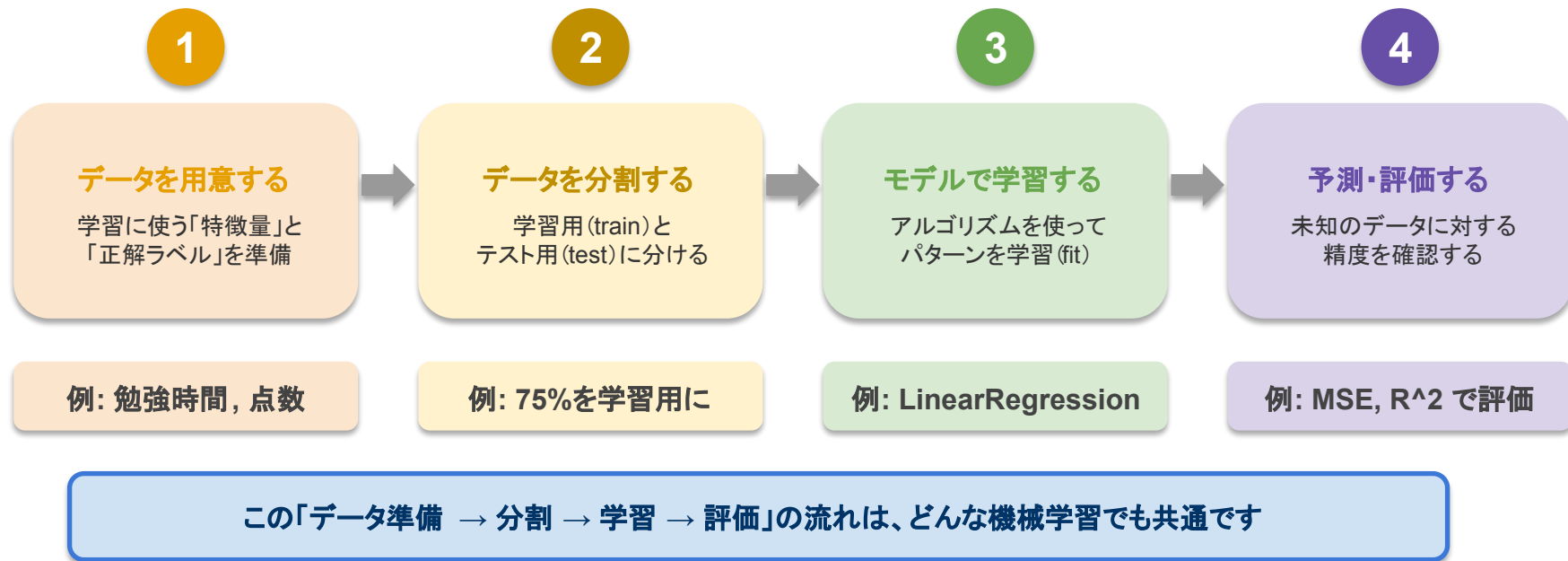
生成AI

文章・画像・音声などを新しく
生み出す。ChatGPT が代表例

機械学習の3つの種類

種類	説明	例
教師あり学習	正解データ付きで学習する	合否予測、価格予測
教師なし学習	正解なしでパターンを発見	顧客グループ分け
強化学習	試行錯誤で最適な行動を学習	ゲームAI、ロボット制御

機械学習の基本的な流れ



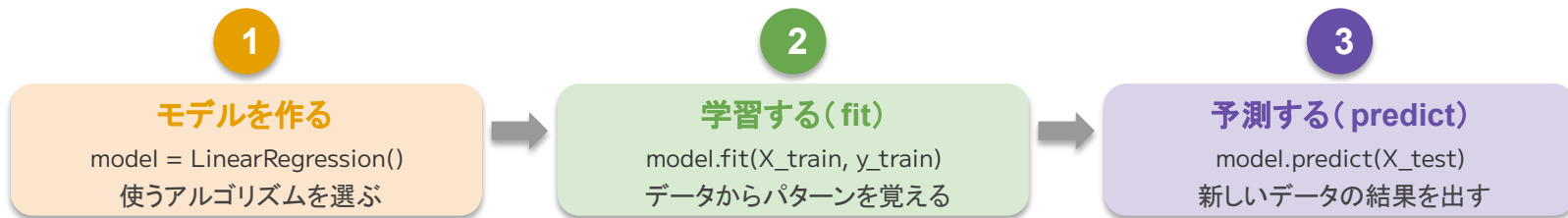
scikit-learnで機械学習を体験

scikit-learn (サイキットラーン) とは

- Pythonで機械学習を行うための定番ライブラリです



scikit-learn の統一された3ステップ



※ どのアルゴリズムでもこの3ステップは同じです。まずはこの流れを覚えましょう

データの準備と分割

- 学習用データとテスト用データに分けます (train_test_split)

```
from sklearn.model_selection import train_test_split
import numpy as np
```

```
# 特徴量 (勉強時間)
```

```
X = np.array([1, 2, 3, 4, 5, 6, 7, 8]).reshape(-1, 1)
```

```
# 正解ラベル (テスト点数)
```

```
y = np.array([30, 40, 50, 55, 65, 70, 80, 85])
```

```
# 学習用75%、テスト用25%に分割
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0)
```

※ reshape(-1, 1) は 1列の2次元配列に変換する操作です

線形回帰モデルで予測する

- 「勉強時間 → テストの点数」を予測するモデルを作ります

```
from sklearn.linear_model import LinearRegression

# モデルを作成
model = LinearRegression()

# 学習 (fit)
model.fit(X_train, y_train)

# 予測 (predict)
y_pred = model.predict(X_test)
print(y_pred)
```

機械学習の3ステップ

1. モデルを作る
2. fit() で学習
3. predict() で予測

演習: 予測モデルを作ってみよう

- 以下のデータを使って、身長から体重を予測するモデルを作しましょう

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import numpy as np

# 身長(cm)
X = np.array([155, 160, 165, 170, 175, 180, 185]).reshape(-1, 1)
# 体重(kg)
y = np.array([50, 55, 58, 65, 68, 72, 78])

# 問題: train_test_split で分割し、
#       LinearRegression で学習・予測してください
```

演習: 回答

- 正解は以下のとおりです

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.25, random_state=0)
```

```
model = LinearRegression()  
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)  
print("予測値:", y_pred)  
print("実際の値:", y_test)
```

モデルの評価

モデルの精度を確認する

- 予測がどれくらい正確かを数値で評価します

```
from sklearn.metrics import mean_squared_error
```

```
# 平均二乗誤差 (MSE) : 小さいほど良い  
mse = mean_squared_error(y_test, y_pred)  
print("MSE:", mse)
```

```
# 決定係数 ( $R^2$ ) : 1に近いほど良い  
score = model.score(X_test, y_test)  
print("R^2:", score)
```

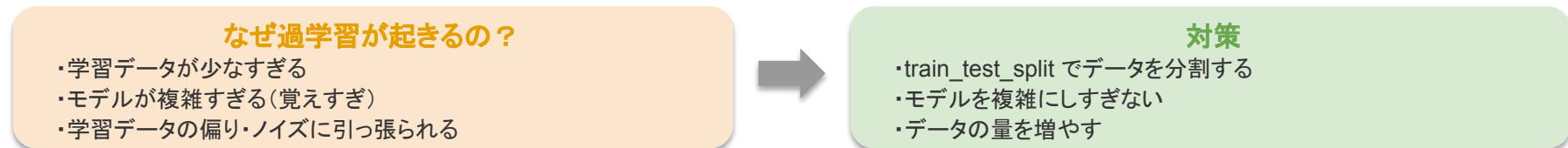
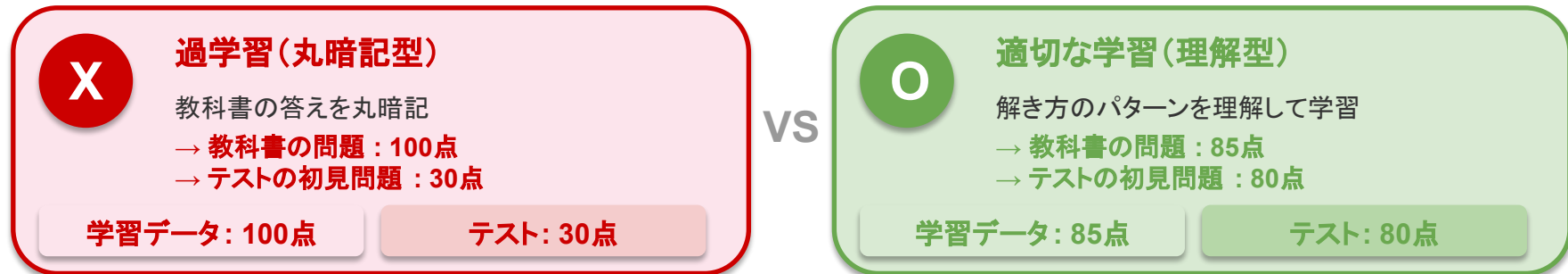
評価指標

MSE: 誤差の平均 (小さい=良い)

R^2 : モデルの説明力 (1=完璧)

過学習(オーバーフィッティング)とは

- 学習データに「合わせすぎる」と、新しいデータに対応できなくなります



だから train_test_split でデータを分けて、テストデータで本当の実力を測ることが大切！

まとめ

NumPy

- ・ 数値計算用のライブラリ
- ・ `np.array()` で配列を作成
- ・ 要素ごとの一括演算が可能
- ・ `np.mean()`, `np.max()` 等で統計量を計算

Pandas

- ・ 表形式データ用のライブラリ
- ・ `pd.DataFrame()` で表を作成
- ・ `df.head()`, `df.describe()` でデータを確認
- ・ 条件でフィルタリングが可能

機械学習

- ・ データからパターンを学習
- ・ `scikit-learn` で実装
- ・ `fit()` → `predict()` の流れ
- ・ `train_test_split` で分割
- ・ 評価指標で精度を確認

```
import numpy as np
import pandas as pd
from sklearn.linear_model import
LinearRegression
from sklearn.model_selection import
train_test_split
```

```
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

機械学習の流れ

1. データを用意する
2. train / test に分割する
3. モデルを作成して学習 (fit)
4. 予測 (predict) する
5. 評価指標で精度を確認する